

Scientific Computing & Advanced Programming Lab

SAURAV SAMANTARAY

Department of Mathematics
IIT Madras

Lecture-1



sauravsray@iitm.ac.in

Welcome!

-
- ▶ Slides will be available at "saurav..."
 - ▶ Codes corresponding to each lectures will be available at ""



Prerequisites and Demands

Prerequisites

- ▶ Some python programming experience
- ▶ Basic linear algebra and calculus

Demands

- ▶ honesty: don't cheat yourself !
- ▶ clocking numerous hours per week in writing / testing codes.



Course Policies

Collaboration

- ▶ struggle first
- ▶ discuss specific doubts over the macroscopic problem
- ▶ don't copy anyone's code

LLMS Usage

- ▶ don't use to solve the problem
- ▶ may be only ask for clarifications

Assignments

- ▶ No assignments
- ▶ A lot of problem sets



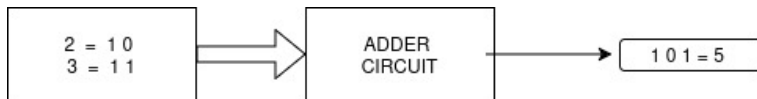
Grading

- ▶ 4 surprise quizzes, 10 Marks each.
- ▶ 1 viva exam, 20 Marks.
- ▶ Class participation: 20 Marks.
- ▶ A final project worth 20 Marks.



Computing

- ▶ What do computers do ?
- ▶ How do they compute ?
 1. Data movement: read / store
 2. logical : Compare / And / Or
 3. add / subtract / Multiply / Divide / Increment / Decrement
 4. control flow: Jump to another instruction
- ▶ Example-1: Calculate $2 + 3$:



- ▶ By itself not very impressive.



Computing

Example-2:

1. Read marks of next student;
 2. Add score;
 3. Compute Avg;
 4. Store result;
 5. Repeat steps for every student;
 6. Print the final grade list.
- ▶ the computer doesn't learn any new operation for each of the instructions
 - ▶ each operation is broken down to basic operation blocks, performed in some sequence



What is programming?

- ▶ Programming is the process of organising basic operations into simple applications which combine to solve a much larger problem.
- ▶ Complex software is built by combining numerous of these tiny, simple instructions into a carefully organised sequence.
- ▶ can compute anything using 6 primitives (Turing)
- ▶ programming language provides a set of primitive operations
- ▶ create "expressions" which are complex but legal combination of primitives
- ▶ write down a sequence of declaration / evaluation usually to meet some end goal.



What is a program?

Program

A program is a sequence of definitions and commands

- ▶ definitions **evaluated**
- ▶ commands **executed**

Scripts/Shells

- ▶ can be typed directly in a shell
- ▶ typically stored in a file



Where things may go wrong

Syntactic errors:

- ▶ common
- ▶ easily caught by compilers
- ▶ English:
 - “sat cat mat” → not syntactically valid
 - “cat sat on the mat” → syntactically valid
- ▶ Programming Language:
 - “hi” 5 → not syntactically valid
 - $3.2 * 5$ → syntactically valid



Where things may go wrong

Static semantic errors:

- ▶ syntactically valid strings
- ▶ no semantic errors
 1. program crashes, stops running
 2. program runs forever
 3. program gives an answer but different than expected



Content

Objects

Branching and Iterations



OBJECTS

- ▶ programs manipulate data objects
- ▶ objects have a type
 - ▶ Saurav is a human so he can breathe, sleep, etc.
 - ▶ An uncle is a relative, so he can give career advice, discuss politics, ask about your grades.
 - ▶ Mango is a fruit, so it can be ...
- ▶ Objects are:
 1. Scalar (cannot be subdivided)
 2. non-scalar (have internal structure that can be accessed)



Scalar objects

- ▶ `int` - represents **integers**, e.g. 7
- ▶ `float` - represents **real numbers**. e.g. 3.14159265
- ▶ `bool` - represents **real Boolean values** True or False
- ▶ `NoneType` - **special** and has one value, None
- ▶ use `type (..)` to see the type of an object



Scalar objects

Type conversion (cast)

- ▶ can convert object of one type to another
- ▶ `float (3)` converts integer 3 to float 3.0
- ▶ `int (3.9)` truncates float 3.9 to integer 3

Print to console

- ▶ `3 + 2`
- ▶ `print (3 + 2)`



Variables

Binding Variables and values

- ▶ `interest = 1000.0 * 9.8 * 6 / 100.0`
- ▶ `roi = 9.8`
- ▶ `prin = 1000.0`
- ▶ `time = 6`
- ▶ `interest = prin * roi * time / 100.0`
- ▶ can modify interest by modifying any of the basic components.



Strings

- ▶ Many programmers deal with text on a daily basis
- ▶ example
 - ▶ web developers work with text that gets input from web forms
 - ▶ Data scientists process text to extract data and perform things like sentiment analysis, to classify opinions in a body of text.
- ▶ Collections of text in Python are called strings



Strings

- ▶ letters, special characters, spaces, digits
- ▶ enclose in

```
hi = 'hello there' or
```

```
hi = "hello there" or
```

- ▶ **concatenate** strings

```
name = "saurav"
```

```
greet = hi + name
```

```
greeting = hi + " " + name
```

- ▶ **String Indexing**

```
>>> flavor = "apple pie"
```

```
>>> flavor[1]
```

```
'p'
```

- ▶ In Python—and most other programming languages—counting always starts at zero.



Strings

- ▶ To get the character at the beginning of a string, you need to access the character at position 0:

```
>>> flavor[0]
'p'
```

- ▶ **String Slicing**

- ▶ Suppose you need the string containing just the first three letters of the string "apple pie".

```
>>> first_three_letters = flavor[0] +
    flavor[1] + flavor[2]
>>> first_three_letters
'app'
```

- ▶ a portion of a string, called a **substring**

```
>>> flavor = "apple pie"
>>> flavor[0:3]
'app'
```



Strings

Strings Are Immutable

- ▶ we can't change them once we've created them

```
>>> word = "goal"
```

```
>>> word[0] = "f"
```

- ▶ If we want to alter a string, we must create an entirely new string



Input / Output

► output

```
>>> x = 1  
>>> print(3) >>> print(5*text)
```

Interact With User Input

► input

```
>>> text = input("Type anything... ")  
>>> print (text)
```



Content

Objects

Branching and Iterations



Comments

```
import math

principal = 500000
interest = 7.5
duration = 20

r = interest / 12 / 100
n = duration * 12

if r == 0:
    payment = principal / n
else:
    payment = principal * r * (1 + r) ** n / ((1 + r) ** n - 1)

print("Monthly EMI = ", payment, " Rupees")
```



Comments

A person can understand the syntax, but they may wonder:

- ▶ Why is the interest divided by 12 and then by 100?
- ▶ Where did this complicated formula come from?
- ▶ Why is there a special case when $r == 0$?

The code works, but the intent is not obvious.



Comments

```
import math

# Loan details
principal = 500000      # Loan amount in rupees
interest = 7.5          # Annual interest rate (in percent)
duration = 20           # Loan repayment period (in years)

# Convert the annual interest rate into a monthly decimal rate,
# as the EMI formula works with monthly interest.
r = interest / 12 / 100

# Total number of monthly installments.
n = duration * 12

# If there is no interest, each installment is simply an equal
# share of the principal.
if r == 0:
    payment = principal / n
else:
    # Compute the Equated Monthly Installment (EMI) using the
    # standard loan amortization formula used by banks.
    payment = principal * r * (1 + r) ** n / ((1 + r) ** n - 1)

# Display the monthly payment amount.
print("Monthly EMI =", payment, "Rupees")
```



Comments

What the comments add

- ▶ Why the interest rate is converted.
- ▶ What n represents.
- ▶ Why there is a special case for zero interest.
- ▶ Where the formula comes from (the standard EMI formula).
- ▶ What each input variable means.



Comments

```
# Print "Hello, world"  
print("Hello, world")
```

- ▶ No comment is needed in this example.
- ▶ best used to clarify code that may not be easy to understand
- ▶ to explain why something is done a certain way
- ▶ comments should be used sparingly
- ▶ Use comments only when they add value



CONTROL FLOW - BRANCHING



Indentation in Python

What is indentation?

- ▶ Indentation means adding spaces (or a tab) at the beginning of a line.
- ▶ In most programming languages, indentation is used mainly to improve readability.
- ▶ In Python, indentation is part of the language syntax.
- ▶ It tells Python which statements belong together as a block.

```
age = 18

if age >= 18:
    print("Eligible to vote")

print("Program finished")
```

- ▶ In Python, indentation is mandatory, not just a matter of style.



Indentation in Python

Incorrect Indentation

```
marks = 45
```

```
if marks >= 50:  
    print("Marks = ", marks)  
print("Pass")
```

